

Informe Técnico de Arquitectura y Seguridad: ClipSync

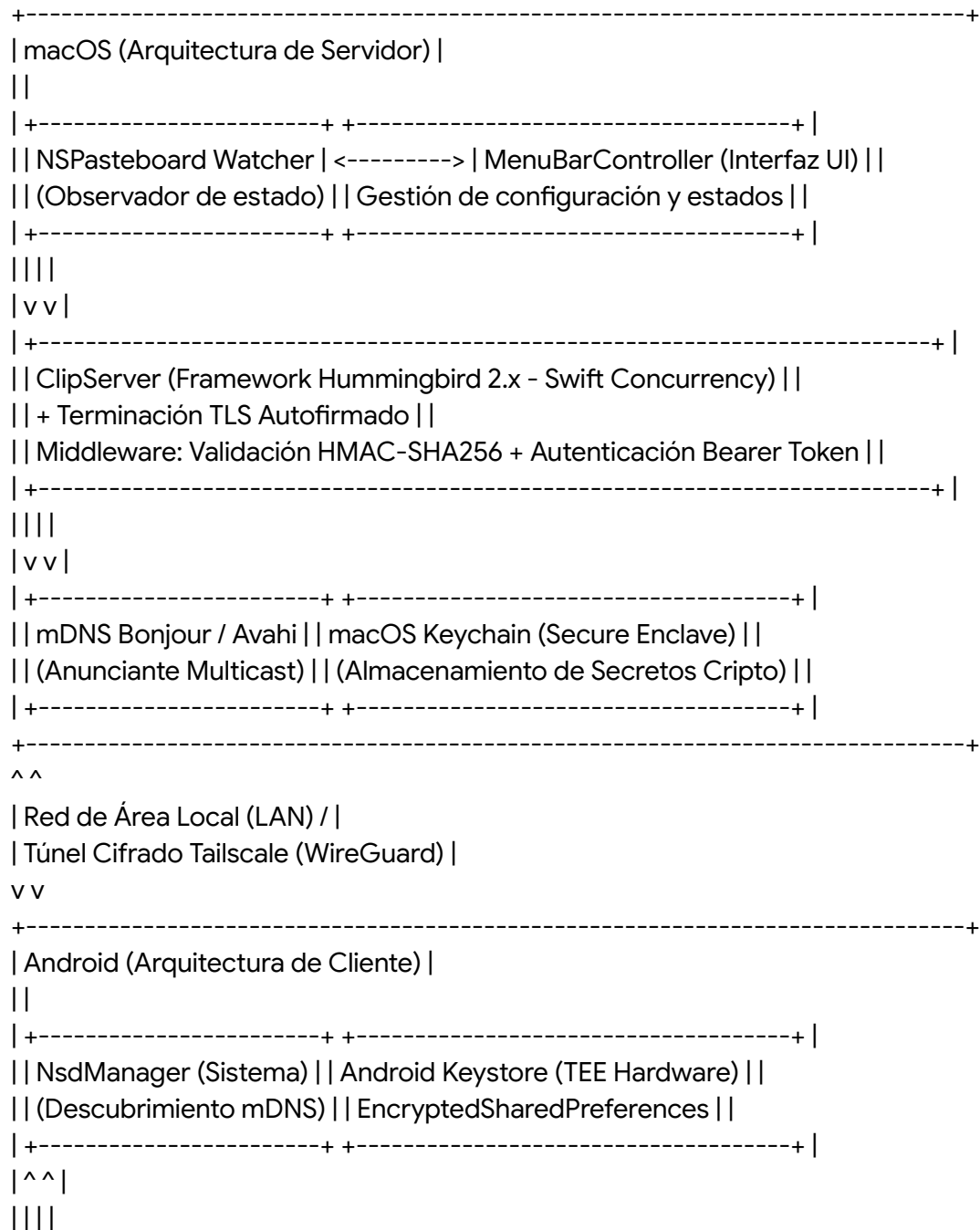
1. Visión general — ¿Qué hace ClipSync y cómo?

El ecosistema moderno de dispositivos informáticos suele fragmentar la experiencia del usuario, siendo el portapapeles uno de los silos de información más restrictivos y frustrantes. La aplicación ClipSync resuelve el desafío de la sincronización de texto e imágenes entre sistemas operativos dispares (macOS y Android) de forma instantánea y criptográficamente segura. Para lograr este objetivo, el sistema descarta los paradigmas convencionales y adopta una arquitectura cliente-servidor invertida: a diferencia del modelo tradicional donde el dispositivo móvil (Android) actúa como servidor pasivo o ambos nodos se conectan a un intermediario en la nube, ClipSync establece la estación de trabajo macOS como el servidor principal ininterrumpido y el dispositivo Android como el cliente efímero.

Esta decisión arquitectónica se fundamenta en las naturalezas radicalmente opuestas de los sistemas operativos en cuanto a la gestión de energía y conectividad de red. Un ordenador Mac suele poseer una dirección IP local predecible, goza de una conexión constante a la red eléctrica o cuenta con baterías de gran capacidad, y su núcleo operativo (macOS basado en Darwin) permite la ejecución persistente de demonios y servidores en segundo plano sin restricciones drásticas. Por el contrario, el ecosistema Android implementa políticas de gestión de batería extremadamente agresivas, particularmente a través de los estados de suspensión profunda conocidos como modo Doze y App Standby. Estas políticas del núcleo de Linux en Android destruyen sistemáticamente los servidores en segundo plano y los sockets de red persistentes para conservar la vida útil de la batería. Al convertir a Android en el cliente, el dispositivo móvil queda exento de mantener puertos de escucha abiertos. Únicamente necesita despertar impulsado por un evento del sistema (como la modificación del portapapeles o la interacción del usuario), iniciar el apretón de manos (handshake) mediante un cliente WebSocket, transferir el contenido de forma asincrónica y permitir que el sistema retorne inmediatamente a un estado de letargo de bajo consumo.

La ausencia deliberada de un servicio en la nube constituye un pilar innegociable en el diseño de esta solución, presentando ventajas e inconvenientes tangibles en el mundo real. La ventaja preeminente es la garantía de una privacidad y confidencialidad absolutas. El portapapeles del sistema es un vector de datos altamente sensible, conteniendo rutinariamente contraseñas en texto claro, tokens de autenticación de doble factor, direcciones físicas, números de tarjetas de crédito y fragmentos de código fuente confidencial. Al no depender de servidores de terceros ni de bases de datos centralizadas en la nube, se elimina por completo el riesgo de exfiltración de datos a causa de brechas en la infraestructura corporativa de intermediarios. Ningún tercero, atacante o entidad gubernamental puede interceptar los datos en reposo en un servidor ajeno. Adicionalmente, la sincronización confinada a la red de área local (LAN) ofrece una latencia de red microscópica, a menudo cifrada en un solo dígito de milisegundos, una

velocidad de reacción inalcanzable en arquitecturas tradicionales que requieren viajes de ida y vuelta a centros de datos geográficamente distantes. El inconveniente técnico principal de este enfoque local es la falta de descubrimiento y enrutamiento predeterminado cuando los dispositivos no comparten el mismo dominio de difusión. Sin embargo, este obstáculo geográfico se puede solventar de manera elegante y segura mediante la integración de redes privadas virtuales de topología de malla, un aspecto que se abordará en profundidad. A continuación, se detalla el diagrama arquitectónico completo del ecosistema, ilustrando el flujo de datos de extremo a extremo y la disposición de los componentes de seguridad y conectividad:



```

| +-----+ |
| | ClipClient (Implementación OkHttp / WebSocket en Kotlin) | |
| | Responsabilidades: Generación HMAC, SPKI Pinning, Debouncing | |
| +-----+ |
| | | |
| | v v v |
| +-----+ +-----+ +-----+ |
| | Shizuku (Capa AIDL) | | SendClipActivity (Capa) | | Access. Service | |
| | UserService Privilegiado | (Actividad Transparente) | | Polling Secundario | |
| +-----+ +-----+ +-----+ |
| | | |
| +-----+ +-----+ +-----+ |
| | |
| | v |
| +-----+ |
| | ClipboardManager (Framework Android) | |
| +-----+ |
+-----+

```

2. Descubrimiento de dispositivos — ¿Cómo se encuentran Mac y Android?

Para que el cliente Android pueda iniciar un apretón de manos con el servidor macOS sin exigir al usuario la fricción de introducir direcciones IPv4 o IPv6 manualmente, el sistema integra el protocolo mDNS (Multicast DNS), comercializado de forma nativa en el ecosistema Apple como Bonjour. El mecanismo subyacente de mDNS opera emitiendo y escuchando datagramas a través del protocolo de transporte UDP, específicamente dirigido a direcciones IP de multidifusión reservadas (como 224.0.0.251 para IPv4) en el puerto estándar 5353.

Empleando una analogía del mundo real, el protocolo mDNS actúa como el pregonero en la plaza pública de un poblado (la red local). En lugar de depender de un directorio telefónico centralizado (un servidor DNS tradicional administrado por un enrutador), el cliente Android alza la voz hacia todos los presentes simultáneamente preguntando: "¿Existe algún servicio del tipo ClipSync disponible en este recinto?". El ordenador Mac, cuyo demonio de red está escuchando atentamente estos gritos de multidifusión, responde directamente proclamando: "Soy el servidor ClipSync, mi identidad de host es MacBook-Pro.local y estoy disponible en el puerto 8443".

Una de las decisiones de ingeniería de seguridad más sofisticadas en la etapa de descubrimiento de ClipSync es la explotación semántica de los registros TXT de Bonjour.¹ El estándar mDNS permite incluir metadatos estructurados como pares clave-valor dentro del paquete de respuesta. El servidor macOS aprovecha esta característica para empaquetar el "fingerprint" (la huella digital criptográfica basada en el algoritmo SHA-256) de su certificado TLS autofirmado dentro de la propia respuesta de anuncio de red.⁴ Cuando el subsistema

Android procesa la respuesta, no solo extrae los vectores de enrutamiento (IP y puerto), sino que asimila anticipadamente la identidad criptográfica estricta del servidor al que está a punto de conectarse. Esta inyección de metadatos prepara el terreno operativo para un anclaje de claves (SPKI pinning) dinámico y hermético, neutralizando de facto cualquier intento de suplantación de identidad (spoofing) o redirección desde el primer milisegundo de la conexión de capa de transporte.

A pesar de su elegancia en entornos LAN aislados, el protocolo mDNS colapsa ante topologías de redes superpuestas modernas como las Redes Privadas Virtuales (VPN) de malla. Tailscale, la solución de interconexión elegida como paradigma para la sincronización remota fuera del hogar, es un exponente de este problema. Tailscale orquesta túneles cifrados de extremo a extremo utilizando el protocolo WireGuard, proporcionando a cada dispositivo un espacio de direccionamiento IP estático e inmutable (típicamente bajo el bloque reservado 100.64.0.0/10).⁵ Sin embargo, por diseño arquitectónico y consideraciones de eficiencia de ancho de banda, Tailscale opera puramente con enrutamiento de capa 3 (Capa de Red en el modelo OSI).⁶ El tráfico de multidifusión UDP, que es el latido fundamental de mDNS, depende intrínsecamente del dominio de difusión de la capa 2 (Capa de Enlace de Datos) de una red física local.⁶ En consecuencia, los paquetes mDNS de Android son descartados por el adaptador virtual tun0 de la VPN; el grito del pregonero no cruza el túnel.⁷

Para franquear esta barrera topológica, ClipSync incorpora un mecanismo de contingencia (fallback) basado en la sincronización manual de IP. El usuario puede introducir estáticamente la dirección IPv4 de la interfaz de Tailscale del Mac. Dado que en este flujo no se realiza el descubrimiento mDNS (y por ende no se obtiene la huella digital del registro TXT previamente), el sistema realiza una comprobación cruzada rigurosa (extra check) durante el apretón de manos manual para garantizar que el terminal que responde coincide criptográficamente con el secreto de emparejamiento almacenado y expone los protocolos esperados, previniendo falsos positivos o conexiones con servicios ajenos que ocupen el mismo puerto en la red virtual.¹¹

El uso de Tailscale representa el paradigma más robusto y limpio para la sincronización en movilidad. Su adopción elimina por completo la necesidad de exponer el Mac directamente a la internet pública (lo que atraería escaneos de puertos maliciosos y bots de fuerza bruta) o de configurar complejas redirecciones de puertos (Port Forwarding) en enrutadores domésticos, manteniendo la inviolabilidad perimetral del sistema.

3. Emparejamiento TOFU — El primer apretón de manos

El proceso crítico de establecimiento de confianza entre la arquitectura móvil y la estación de trabajo se sustenta en un modelo de validación de seguridad denominado TOFU (Trust On First Use o Confianza en el Primer Uso). A diferencia de las infraestructuras convencionales de clave pública (PKI) que dominan el ecosistema web actual —donde una Autoridad de Certificación (CA) de terceros actúa como notario garantizando la identidad de los servidores—, el paradigma TOFU asume una confianza plena pero restrictiva en el momento inicial de la

interacción, cristalizando esa identidad criptográfica para rechazar de plano cualquier discrepancia en el futuro.¹²

La analogía conductual del modelo TOFU es la de entablar contacto visual directo con una persona por primera vez y memorizar meticulosamente sus rasgos faciales biológicos. No se requiere que el gobierno civil (la Autoridad de Certificación) expida un documento certificando quién es esa persona; simplemente se acepta que el individuo con el que se interactúa hoy en el entorno seguro del propio hogar es la misma entidad con la que se interactuará mañana. Si en un encuentro futuro aparece una persona diferente, aunque porte una placa gubernamental válida afirmando ser la identidad original, la discrepancia facial delatará de forma irrefutable que se trata de un impostor.

El flujo transaccional de emparejamiento, orquestado a través de una secuencia de estados deterministas, se ilustra a continuación:

Cliente Android (Iniciador) Servidor macOS (Autoridad Local)

```
||
| 1. Petición Multidifusión UDP (mDNS) |
|----->|
||
| 2. Respuesta mDNS (IP, Puerto, TXT: SHA256) |
|<-----|
||
| 3. Solicitud de inicio de emparejamiento |
| (HTTP POST /api/pair/start) |
|----->|
||
| 4. macOS genera un código de 6 dígitos |
| (Generador Pseudoaleatorio Cripto) |
| con Tiempo de Vida (TTL) de 300s. |
| Se renderiza en la pantalla del Mac. |
||
| 5. El usuario humano actúa como canal |
| fuera de banda (Out-Of-Band), lee el |
| código en Mac y lo teclea en Android. |
||
| 6. Validación HTTP POST /api/pair/verify |
| Payload: { "code": "849201" } |
| (Canal cifrado vía TLS 1.3 con SPKI pin) |
|----->|
||
| 7. macOS verifica el código. Si es |
| válido, emite dos secretos atómicos: |
| - Bearer Token (Autenticación API) |
| - Pairing Secret (Clave para HMAC) |
|<-----|
```

||

| 8. Android confina los secretos en el |

| Hardware TEE (Android Keystore). |

||

El código de autenticación de 6 dígitos exhibido en la interfaz del Mac no es trivial; es una construcción criptográficamente aleatoria con un Tiempo de Vida (TTL) efímero de exactamente 300 segundos, diseñado como un desafío de un solo uso (nonce). La entropía intrínseca de una cadena numérica de 6 dígitos contempla exactamente un millón de permutaciones posibles (desde 000000 hasta 999999). Aunque desde una perspectiva teórica un millón de combinaciones es un espacio de claves ínfimo susceptible a ataques de fuerza bruta computacionales¹⁴, el vector de tiempo (TTL) neutraliza esta debilidad matemáticamente de manera absoluta.

Para que un atacante informático consiga adivinar el PIN correcto iterando combinaciones dentro de una ventana de 5 minutos, requeriría emitir y procesar más de 3.333 solicitudes de red cifradas por segundo, una tasa de tráfico anómala que desencadenaría de inmediato alertas en cualquier demonio de red y que el servidor ClipServer mitiga imponiendo limitaciones de tasa (rate limiting) estrictas a nivel del enrutador HTTP.¹⁶ Por lo tanto, el temporizador expira y destruye la semilla criptográfica en la memoria del Mac órdenes de magnitud antes de que el adversario logre barrer una porción estadísticamente relevante de las claves posibles.¹⁸

¿Qué ocurriría si un adversario intercepta subrepticamente el código de 6 dígitos durante su ciclo de vida útil, por ejemplo, mediante vigilancia física sobre el hombro del usuario (shoulder surfing)? La propiedad de "un solo uso" del código previene compromisos secundarios. Si el atacante intenta enviar el código tras su expiración, el servidor lo descartará; y si intenta anticiparse al usuario legítimo, causará una desincronización evidente. Peor aún para el atacante, cualquier intento de capturar el código mediante la interceptación del tráfico inalámbrico de la red de área local (Packet Sniffing) es inútil: el código nunca transita en texto claro por el cable. Viaja blindado dentro del túnel asimétrico de la capa TLS, el cual ya ha sido validado contra ataques de intermediario (Man-In-The-Middle) mediante la técnica de SPKI pinning establecida en la fase de descubrimiento.

4. TLS con certificado autofirmado — La capa de cifrado

El protocolo Transport Layer Security (TLS), preferiblemente en su iteración 1.3, constituye el foso de defensa primario del sistema. Operar bajo el protocolo HTTP en texto claro resulta inaceptable desde cualquier estándar de la ciberseguridad contemporánea, incluso cuando la comunicación se confina a los límites físicos de una red doméstica privada. Las redes residenciales e intraempresariales de hoy están saturadas de dispositivos de Internet de las Cosas (IoT) —tales como televisores inteligentes, electrodomésticos conectados o cámaras IP— con vulnerabilidades críticas sin parchear. Un dispositivo IoT infectado en la red local podría pivotar y ejecutar escaneos de paquetes, absorbiendo subrepticamente contraseñas o

tokens expuestos en el tráfico HTTP sin cifrar de ClipSync.

La topología de ClipSync obliga a la comunicación a través de direcciones IP internas o dominios locales terminados en .local. Bajo las normas globales del consorcio de Autoridades de Certificación, es categóricamente imposible obtener un certificado TLS público firmado (como los de Let's Encrypt o DigiCert) para una dirección IP de red privada debido a la incapacidad de probar la propiedad criptográfica y exclusiva del dominio.¹² Frente a esta restricción, la arquitectura dicta que el servidor macOS debe generar y emitir su propio certificado digital (un certificado autofirmado).

Desde la perspectiva matemática de la criptografía, un certificado TLS autofirmado provee exactamente la misma complejidad geométrica y los mismos algoritmos impenetrables de cifrado de bloque que el certificado comercial más caro del mundo.¹² La divergencia radica exclusivamente en la cadena de confianza: el subsistema TLS del dispositivo Android se negará por defecto a establecer una conexión, arrojando una excepción de seguridad, porque el creador del certificado (el propio ordenador Mac) no figura en la lista inmutable de autoridades certificadoras aprobadas que Google incorpora en el sistema operativo Android.¹⁹

Para obligar al cliente a aceptar este escudo criptográfico sin claudicar ante fallas de seguridad, ClipSync despliega una técnica avanzada denominada SPKI Pinning (Subject Public Key Info Pinning). En vez de instruir al cliente HTTP (la librería OkHttp en Kotlin) para que eluda las advertencias y confíe ciegamente en todos los certificados no firmados (una práctica irresponsable común en el desarrollo amateur), el sistema aísla y extrae la clave pública asimétrica codificada en ASN.1 contenida en la carga útil del certificado del Mac.⁴ Acto seguido, calcula su entropía digestiva utilizando el algoritmo de hash SHA-256.¹ Durante el apretón de manos inicial del socket TLS, antes de que fluya ningún byte de la capa de aplicación, el cliente Android compara este hash del servidor entrante contra el hash que adquirió con anterioridad del registro TXT de mDNS. Si ambas firmas de 256 bits colisionan de manera perfecta, el túnel cifrado se sella y autoriza. Si discrepan en un único y solitario bit matemático, el socket se derriba sin contemplaciones.

El anclaje SPKI es vastamente superior a depender de la infraestructura de autoridades de certificación del sistema.⁴ Retomando analogías tangibles, permitir que el sistema operativo dicte la confianza basándose en Autoridades de Certificación es el equivalente a franquear el paso a cualquier individuo capaz de presentar un Documento de Identidad Nacional formalmente emitido por un burócrata del gobierno. El SPKI Pinning, por el contrario, requiere que poseas de antemano la fotografía de alta resolución y biometría exacta de la persona específica a la que esperas en la puerta; la procedencia política de su documento carece de valor si la retina no coincide. Esta inamovilidad erradica los ataques MITM (Man-in-the-Middle) de raíz. En un esquema MITM, un atacante posicionado entre el Mac y Android alteraría la tabla de enrutamiento ARP, forzando a los paquetes a atravesar su ordenador. El atacante presentaría un certificado falso para intentar descifrar y recifrar la carga. Al no poseer la clave privada correspondiente a la clave pública exacta esperada por Android, el SPKI Pinning desencadenará un fallo criptográfico fatal, revelando el ataque y abortando el flujo de sincronización.

Para la fundación asimétrica del certificado, ClipSync margina el estándar histórico RSA en

favor de la criptografía de Curva Elíptica (Elliptic Curve Cryptography), concretamente la curva P-256 recomendada por NIST.¹² La justificación técnica subyacente es la economía de escala computacional: una clave de curva elíptica de apenas 256 bits de longitud proporciona una fortaleza criptográfica contra el criptoanálisis equivalente a una mastodóntica clave RSA de 3072 bits.¹² Esta asombrosa reducción geométrica minimiza drásticamente los ciclos de procesador necesarios para efectuar las multiplicaciones asimétricas complejas durante el apretón de manos TLS. En el mundo real del cliente móvil, esto se traduce en una negociación de socket que requiere fracciones de milisegundos y un ahorro palpable en la degradación de la batería del dispositivo Android tras centenares de sincronizaciones de portapapeles diarias. No obstante, se debe demarcar que el escudo TLS, si bien es impenetrable en el cable de red, posee limitaciones semánticas en la capa de aplicación (Capa 7). TLS garantiza que los datos en tránsito estén ocultos contra el espionaje (confidencialidad) e inmunes a la manipulación en red, pero TLS no discrimina la legitimidad moral o la identidad persistente de las transacciones HTTP. Si un actor malicioso extrae el token legítimo de sesión del dispositivo Android vulnerado y establece una conexión directa con el Mac desde otro dispositivo, la capa TLS negociará el cifrado plácidamente. Para blindar las amenazas inherentes a la mutación y procedencia del propio contenido de las solicitudes, el protocolo debe escalar hacia la firma HMAC.

5. HMAC-SHA256 — La firma de cada request

Mientras que la suite criptográfica TLS se encarga de crear un conducto invulnerable en el éter de la red, el código de autenticación de mensajes basado en hash (HMAC) se aplica de forma granular para asegurar ineludiblemente la integridad morfológica, el origen y la vitalidad temporal de cada paquete HTTP y trama WebSocket individual que surca dicho conducto.²⁰ El algoritmo HMAC orquesta una función hash criptográfica (SHA-256) amalgamándola simétricamente con una clave secreta (el secreto de emparejamiento acordado durante el TOFU) para generar una firma alfanumérica única para la carga útil que se está enviando.²⁰ La diferenciación conceptual entre el mero cifrado y la protección de integridad es sutil pero esencial. El cifrado (TLS) funciona como si el texto del portapapeles fuera transportado dentro de un maletín de titanio blindado sin cerradura compartida; nadie a su alrededor puede contemplar los folios de su interior. Sin embargo, el maletín en sí mismo podría ser objeto de alteración física durante el tránsito si no se custodia correctamente. HMAC actúa de manera equivalente al estampado de un pesado sello de cera fundida en cada una de las páginas, impreso mediante un anillo de linaje heráldico único e irrepetible. El sello por sí solo no tiene el propósito de ofuscar el texto (si estuviera a la vista), pero testifica sin sombra de duda la autoría exacta del redactor (autenticidad), revelando instantáneamente la menor rasgadura o falsificación si un intermediario intentara reemplazar una palabra, ya que solo el poseedor del anillo heráldico (el cliente legítimo con el Pairing Secret) puede forjar un nuevo sello válido.²⁰ Una sofisticación imperativa en la matriz de diseño de ClipSync es la obligatoriedad de adjuntar una marca de tiempo cronológica (timestamp) basada en la época UNIX dentro de la cabecera HTTP o la trama JSON, incluyendo intrínsecamente esta marca en el corpus computacional sobre el que se calcula la firma HMAC. Esta inclusión quirúrgica es la respuesta definitiva contra los devastadores Ataques de Repetición (Replay Attacks).²⁰

Un ataque de repetición clásico ocurre cuando un adversario pertrechado con herramientas como Wireshark captura subrepticamente el torrente de datos TLS de la red. Aunque el adversario sea incapaz de descifrar la capa criptográfica para comprender que el paquete contiene la instrucción "copiar la palabra 'admin_password'", el atacante puede inyectar los mismos bytes crudos exactos en la interfaz de red del Mac días más tarde. Ante un sistema ingenuo sin estampas de tiempo, el Mac descifraría el paquete válido y reescribiría el portapapeles inyectando el texto antiguo. Al embeber el timestamp en la matriz HMAC, el motor validador en Swift evalúa la disparidad temporal del mensaje respecto a su reloj atómico local. Si la diferencia sobrepasa un corredor de tolerancia estricta (usualmente una ventana de deriva de ± 60 segundos), el paquete es categorizado como una anomalía caduca y purgado sin contemplaciones. Cualquier conato del atacante por modificar la estampa de tiempo para modernizarla fracasará de forma dramática, ya que al alterar un solo byte de los datos, el hash de validación calculado por el servidor colisionará destructivamente con la firma HMAC original.²²

La dependencia exclusiva del protocolo TLS, excluyendo a HMAC de la ecuación, presentaría una vulnerabilidad estructural insidiosa. Por ejemplo, en infraestructuras donde la terminación TLS se efectúa prematuramente en un proxy inverso (como Nginx o Traefik instalado en el mismo ordenador o servidor doméstico), la conexión entre el proxy y el demonio interno pierde la protección de la capa de transporte; sin HMAC, el tráfico inter-proceso podría ser espiado o manipulado sin levantar alarmas.

Las consideraciones de seguridad permean hasta las capas lógicas más profundas del código. Al comparar las firmas HMAC generadas dinámicamente frente a las suministradas en las cabeceras HTTP, tanto en el marco de Swift como en Kotlin, está terminantemente prohibido utilizar operadores condicionales básicos de igualdad lexicográfica (como `==` o el clásico `String.equals()`).²³ El uso de una comparación estándar constituye una negligencia arquitectónica que expone el algoritmo a Ataques de Temporización (Timing Attacks).²²

Las instrucciones de CPU responsables de ejecutar funciones de igualdad en cadenas habituales están diseñadas para la eficiencia; en el nanosegundo en que perciben una divergencia en un byte específico entre las dos cadenas comparadas, interrumpen la evaluación y devuelven un resultado booleano negativo.²⁵ Un atacante metódico y paciente puede aprovechar esta optimización emitiendo miles de firmas erróneas y midiendo los tiempos de latencia o rechazo a escala de microsegundos. Si el servidor demora marginalmente más en rechazar una solicitud falsificada en particular, el atacante deduce matemáticamente que acertó el primer carácter hexadecimal de la firma real, repitiendo el ciclo de forma lineal hasta desgranar, carácter por carácter, la firma íntegra sin poseer el secreto original.²² Para erradicar este vector microscópico, ClipSync exige la utilización de comparadores de tiempo constante (Constant-Time Comparisons), como la función `CryptoKit.elementsEqual` en el ecosistema Apple o equivalentes probabilísticos en la máquina virtual Java.²¹ Estas funciones recorren inexorablemente cada byte de la cadena evaluada, consumiendo siempre la misma cantidad de ciclos de procesador, enmascarando cualquier desequilibrio temporal y privando al adversario de métricas deductivas operacionales.²⁴

En retrospectiva, la convergencia de estas doctrinas criptográficas forja una defensa en

profundidad en la que ninguna pieza es redundante:

1. **Capa TLS:** Asegura la clandestinidad inescrutable de los datos, cegando a observadores externos.
2. **Capa HMAC:** Sella de manera inmutable el contenido del paquete, proveyendo integridad, autoría confirmada y frescura temporal innegociable.
3. **Capa Bearer Token:** Otorga la prerrogativa lógica de acceso, certificando la elegibilidad del cliente frente a las rutas de la API subyacente.

6. Bearer Token — Autenticación de sesión

El control de los derechos de acceso en la arquitectura transaccional recae en la figura del Bearer Token. Este token, conceptualmente definido como una cadena alfanumérica criptográficamente opaca y carente de significado estructural en sí misma, asume el papel de un salvoconducto diplomático infalsificable. Su etiqueta "al portador" (Bearer) deriva de su semántica: la plataforma asume que la entidad informática que presente válidamente este testigo criptográfico inyectado en los encabezados estándar de la conexión (bajo el formato `Authorization: Bearer <token_string>`) ostenta la credencial y la autoridad necesarias para transitar por las rutas de la API del servidor local. Este estatus diplomático se consolida en una ocasión única, durante el rito del emparejamiento TOFU analizado en párrafos anteriores, y es la constante de autorización exigida en cada latido de los sockets persistentes.

ClipSync incorpora una capa de mitigación defensiva que eleva el Bearer Token más allá de las convenciones clásicas de la industria, previniendo vectores de explotación secundaria en bases de datos: el servicio alojado en la estación macOS se niega en rotundo a archivar la cadena del token real en texto claro dentro de sus registros de memoria permanente. En su lugar, el servidor computa unidireccionalmente la representación entrante utilizando el algoritmo hash SHA-256 y confina exclusivamente este resultado ofuscado.²³

Esta estrategia arquitectónica descabeza una modalidad de asalto devastadora conocida como "Pass-the-hash" o ataque de exposición de base de datos local. Considérese un escenario donde un malware forense o un proceso colateral defectuoso ejecutándose en el entorno del Mac burla el sandboxing y exfiltra el archivo de configuración del servidor. El botín extraído estará desprovisto de valor operativo. Dado que las funciones hash criptográficas son matemáticamente deterministas pero irreversiblemente asimétricas, el atacante no tiene forma empírica de reconstruir el Bearer Token original partiendo del hash capturado. Si el intruso intentase presentar el hash robado de forma directa al servidor como credencial de acceso, la lógica del servidor aplicaría un segundo cálculo hash al valor entrante y compararía un "hash del hash" con la base de datos, lo cual desembocaría invariablemente en un fallo de validación categórico.

Respecto a la neutralización de identidades y revocación de acceso, la topología implementa esquemas que afectan diferentemente al ciclo de confianza. Existe una bifurcación táctica entre invalidar un token singular y revocar el "Pairing Secret" de forma global. La eliminación de un secreto de emparejamiento maestro opera de facto como un botón nuclear que aniquila la raíz jerárquica de la que nacen todas las confianzas. Esta medida draconiana exige, invariablemente, que todo dispositivo periférico repita el protocolo TOFU (escanear y teclear

un código efímero de 6 dígitos) si aspira a reanudar la sincronización, asegurando un borrón y cuenta nueva criptográfico en el entorno local. Por otro lado, la anulación quirúrgica de un Bearer Token individual permite purgar un dispositivo comprometido específico, salvaguardando la operatividad del resto de los nodos inalterados, si bien esta funcionalidad asimétrica representa un reto constante de sincronización de estado persistente.

¿Cuáles son las ramificaciones de que un adversario consiga sustraer y apropiarse de un Bearer Token en estado puro? Si, mediante una brecha en la memoria volátil del cliente Android o explotación forense del dispositivo de almacenamiento, un atacante logra extraer la cadena del token legítimo, poseería las llaves para accionar teóricamente la maquinaria del servidor Mac. Sin embargo, su capacidad de acción en la arquitectura híbrida de ClipSync sería nula. Dado que el atacante carece del secreto HMAC —que reside disociado y es ineludible para generar la firma de integridad de los paquetes—, cualquier comando enviado por el token usurpado sería aniquilado inmediatamente al chocar con las defensas de validación de carga útil del Mac. El intruso se vería forzado a vulnerar simultánea y concertadamente dos anillos de seguridad asilados (Autenticación y Firma de Integridad) para tomar el control logístico del torrente de datos, elevando las barreras al punto de disuadir intentos de nivel no estatal.

7. Almacenamiento de secretos

Un protocolo de enrutamiento invulnerable carece de propósito si el material criptográfico en los extremos (las claves asimétricas, los tokens y las semillas HMAC) yace a merced de extracciones locales o vulnerabilidades a nivel de disco físico. Para fortificar las llaves maestras, ClipSync subroga esta responsabilidad a los subsistemas avanzados de encriptación atrincherados en los entornos de hardware criptográfico integrado en el silicio de cada sistema operativo anfitrión.

Arquitectura y Plataforma	Marco Subyacente	Mecánica de Protección y Silicio
Ecosistema Apple (macOS)	Keychain Services	Repositorio estructurado en bases de datos SQLite y resguardado por el sistema operativo. En ordenadores equipados con la arquitectura Apple Silicon o el chip T2, la generación, envoltura y protección final de la clave de encriptación del Keychain se ampara físicamente en el Secure Enclave Processor (SEP) . ³⁰ Este es un coprocesador de seguridad, desconectado algorítmicamente del

		procesador de aplicaciones, que corre su propio sistema operativo (L4 microkernel modificado) y un Acelerador Criptográfico de Hardware (AES Engine) de alto rendimiento resistente a análisis de energía.³⁰ Los datos críticos se ciñen de por vida al Identificador Único Universal (UID) incrustado en los fusibles del chip.³⁰ Las llaves no son accesibles al sistema maestro de macOS (sepOS); el sistema solo remite peticiones a la bóveda oscura, impidiendo exfiltraciones por compromiso de memoria volátil. Si un atacante extrajera los módulos de almacenamiento SSD flash e intentase leerlos en una estación forense, la ausencia del UID atado al hardware subyacente y la falta del PIN imposibilitaría categóricamente el descifrado a nivel de bloques.³⁰
Ecosistema Móvil (Android)	Android Keystore + EncryptedSharedPreferences	Combina una capa de envoltura XML asimétrica donde tanto las claves declaradas como los contenidos de valor están completamente cifrados antes de persistir en el disco lógico. Las llaves simétricas maestras encargadas de blindar este componente se anidan irremisiblemente dentro del Android Keystore, un sistema que delega la validación atómica y las operaciones matemáticas al Trusted

		Execution Environment (TEE) (comúnmente implementado mediante tecnologías como ARM TrustZone) o un hardware especializado a prueba de manipulaciones físicas (Secure Elements/Titan M). ³² Dentro del TEE, incluso si una cepa de código malicioso llegara a ganar compromisos absolutos del núcleo del sistema operativo Android, la extracción directa del material de las llaves en claro resultaría tecnológicamente inalcanzable, ya que la memoria RAM del TEE es físicamente inaccesible para el kernel del teléfono. ³⁴
--	--	---

Threat Model Analítico de los Subsistemas de Almacenamiento Criptográfico:

Modelo de Ataque / Vector Amenazante	Grado de Resistencia	Razonamiento Técnico Inherente
Ejecución de App Maliciosa en Sandboxing Local	Altísima	Los sistemas operativos contemporáneos emplean esquemas rígidos de Control de Acceso Obligatorio (MAC), como SELinux y sandboxing de memoria por procesos y asignación de identificadores UID Linux individuales. ³⁷ Una aplicación fraudulenta (e.g., linterna) alojada en el dispositivo es repelida físicamente al tratar de saltar al espacio de memoria que encapsula a ClipSync y sus repositorios Keystore.
Clonación o Extracción Forense del Disco Físico Apagado	Extrema	Si el móvil Android está apagado o bloqueado en un estado pre-arranque (Before First Unlock), la protección

		criptográfica (File-Based Encryption) blinda la unidad.³⁶ Sin proveer la contraseña alfanumérica, el patrón de entropía o el vector biométrico atados al Secure Enclave o TrustZone, descifrar los bloques exfiltrados es un problema matemático irresoluble fuera del alcance computacional moderno.³⁰
Infección Persistente de Root o Escalada de Privilegios Kernels	Media / Baja	Una amenaza con derechos su o permisos de root absolutos dentro del espacio Android activo se topa con el blindaje del TEE para leer la llave.³⁴ Sin embargo, la amenaza asimila permisos para operar como un oráculo de descifrado, es decir, puede usurpar el rol de la aplicación ClipSync obligando al TEE a descifrar mensajes e inyectando binarios precompilados en el espacio de memoria viva (RAM hook), saltándose la seguridad periférica.
Vulneración de Interfaz Humana (Ataque a un Terminal Desbloqueado)	Nula	El modelo arquitectónico hardware capitula intrínsecamente si un adversario obtiene acceso al móvil físico mientras este permanece en un estado desbloqueado y con las llaves de descifrado cargadas y operativas en la memoria viva para la ejecución de la app principal; el hardware, por definición, acata ciegamente al operador presente bajo el presunto de identidad validada.

8. El problema del portapapeles en Android — La

barrera más difícil

El diseño de ClipSync está dominado ineludiblemente por las políticas agresivas y oscilantes del ecosistema Android respecto al aislamiento e inviolabilidad de los datos alojados en el portapapeles temporal del sistema. El vector de ataque que orilló a Google a iniciar una escalada defensiva fue el historial persistente de aplicaciones parasitarias —herramientas utilitarias menores o juegos maliciosos publicadas en la Google Play Store— que abusaban sin restricciones de los oyentes del portapapeles para minar sigilosamente en segundo plano.³⁹ Al ejecutarse en el dispositivo, estos parásitos se dedicaban a extraer el material de copiado y pegado cada segundo para robar carteras de criptomonedas, credenciales bancarias generadas por gestores de contraseñas, o espiar correspondencia privada corporativa sin que el usuario observara ninguna manifestación anómala en la interfaz.³⁹

Evolución Cronológica y Restricciones a Nivel de Kernel/API:

- **Android 10 (API Nivel 29 / Q):** Constituyó un cambio de paradigma monolítico. A partir de esta iteración, Google implantó un cerco rígido y sistémico donde el acceso al portapapeles para los componentes de software subyacentes se bloqueó categóricamente. Únicamente se otorgaría permiso a la interfaz si esta ocupaba el rol predefinido de editor de método de entrada por defecto (teclados virtuales o IME, por sus siglas en inglés) o si el proceso que efectuaba la petición detentaba un control primario y activo con el foco de la pantalla de usuario.³⁹ Los oyentes pasivos morían instantáneamente en contextos de segundo plano y los intentos de invocación devolvían nulo.
- **Android 12 (API Nivel 31 / S):** Las restricciones se engrosaron con vectores de diseño disuasorios basados en retroalimentación inmediata (Visual Feedback). Si una app activa y en pantalla se aventuraba a inspeccionar o acceder a contenidos recién volcados al portapapeles cuyo origen subyacente procediese de una app vecina ajena, el sistema proyectaría un mensaje compulsivo superpuesto y persistente del tipo "Toast" para exponer y humillar tal comportamiento no previsto y concienciar de forma inmediata al operador humano (ej. "ClipSync copió texto del navegador").³⁹ Esta alerta sistemática es mandataria en la base de Android y desincentiva enormemente su uso subrepticio.⁴⁵
- **Android 13+ (API Nivel 33 / Tiramisu):** Elevó la protección hacia la aserción y transparencia inquebrantables, anclando permanentemente notificaciones de interfaz de baja altura superpuestas (Overlay Clipboard Visualizers) durante transferencias pasivas e implementando mecanismos agresivos para cortar accesos paralelos que todavía explotaban fallos pasivos para escrudñar transferencias temporales sin autorización manifiesta.⁴⁶

Encontrar una panacea que brinde una sincronización en segundo plano ininterrumpida frente a un modelo de madurez tan restrictivo resulta imposible. ClipSync despliega entonces tres vectores superpuestos o capas de contingencia funcional que sortean este impedimento a base de recursividad de la interfaz y explotación de privilegios alternativos.

Capa 1: Shizuku — El truco del shell privilegiado

El entorno Android Debug Bridge (ADB) fue estructurado a nivel fundacional como un puerto de comandos y depuración interactivo dirigido a analistas de sistemas, programadores de código e ingenieros de calidad de software.⁴⁸ Por concepción nativa, un cliente conectado al sistema centralizado de demonios ADB dispone de permisos de un rango marcadamente superior al resto de los binarios sandboxeados instalados en el terminal. Shizuku materializa la elevación de estos privilegios. Empleando los terminales locales de depuración inalámbrica (Wireless Debugging) sobre puertos dinámicos nativos, Shizuku permite la orquestación e inyección controlada del subconjunto de API autorizadas por ADB, trasladándolas directamente como un túnel para las aplicaciones finales, obviando de modo elegante la intrincada e invasiva necesidad de que el consumidor realice el enraizamiento absoluto o root a nivel de superusuario de su dispositivo.⁴⁸

Para consumir la fusión interactiva con ClipSync, se implementa una infraestructura técnica bajo el Android Interface Definition Language (AIDL). Este entorno proporciona los bloques lógicos para el tránsito de procesos intercomunicados (IPC), donde múltiples procesos o espacios aislados convergen en el intercambio asíncrono de punteros u objetos Java fuertemente tipificados bajo un contrato de interfaz común.³⁷ ClipSync introduce así un componente UserService remoto que se desacopla e inyecta directamente al seno del esqueleto del demonio residente de Shizuku.⁴⁸

Desde este mirador perimetral de jerarquía privilegiada, el UserService interactúa y evalúa el servicio ClipboardManager vistiendo el estatus intocable del sistema base, por lo que evade holgadamente toda muralla de detección o represión preestablecida por las iteraciones API. La máquina de flujo de vida útil interna (State Machine) transita invariablemente a través de eslabones vitales como NOT_INSTALLED y el letargo intermitente por políticas de hibernación o de muerte térmica dictadas en Android originando fallos severos tipo NOT_RUNNING o el bloqueo terminal NO_PERMISSION.⁵² Si se supera con validaciones, penetra en el estado de negociación BINDING antes de aterrizar y converger en la fase plenamente funcional u operativa catalogada de READY y lidiar posteriormente con fallos inesperados de extinción con el parámetro de control asíncrono que fuerza la invocación vital de destroy() del AIDL para liberar rastros sucios a través de un código transaccional imperativo (fijado a 16777114 o equivalentes en el esqueleto base).⁴⁸

Su adopción práctica resulta análoga al supuesto de contratar a un delegado administrativo que, al portar los salvoconductos universales (privilegios nativos de depurador ADB), es encomendado a transitar el edificio (el sistema de librerías del teléfono). Una vez que el gerente propietario (el usuario mediante el setup visual e inalámbrico del desarrollador) provee de credencial a dicho delegado, la tarea de escanear los faxes entrantes y salientes (portapapeles) ocurre a espaldas de la mirada del supervisor general y los sistemas restrictivos ordinarios.⁴⁹

Capa 2: Accessibility Service — El polling como fallback

La segunda trinchera (capa de contingencia de rescate) atiende a sistemas de legado (dispositivos congelados en API Nivel 30 o Android 11). ClipSync, a modo de último recurso heurístico, aprovecha la ventana subyacente que confieren los Servicios de Accesibilidad

(AccessibilityService).⁵⁵ Originalmente destinados para la inclusión tecnológica de colectivos con diversidad funcional, esta capa API de nivel de sistema inyecta lectores y narradores robóticos en la composición visual para auditar de forma perenne la información expuesta. Dado que una orden pasiva o estática y eficiente bajo consumo (OnPrimaryClipChangedListener) generaría ecos mudos sin retornos, la aplicación adopta la técnica agresiva del interrogatorio cíclico o asíncrono en bucle constante (Polling Framework), estructurado mediante llamadas repetitivas Handler.postDelayed. A un compás iterativo cadencioso de 500 milisegundos de silencio intercalado, el nodo lanza un disparo al corazón de la placa para rescatar todo contenido y contrastarlo en bloque contra la copia interna previamente salvada. Para frenar colapsos sistémicos graves donde se sature el canal RAM tras la copia iterativa y sucesiva de páginas masivas HTML o megabytes en texto sin procesar, la comprobación se refina empleando entropía criptográfica (hashing comparativo ciego) en vez del costoso procedimiento analítico byte a byte.

Sin embargo, las vulnerabilidades latentes inherentes a esta capacidad de lectura y manipulación integral (que permitirían simulaciones en red ocultas de pulsaciones táctiles por parte de programas o malware de robo bancario como BrasDex o Xenomorph) alarmaron tan profundamente al ecosistema central de Google ⁴⁰ que forzaron bloqueos paralelos. Debido al agotamiento drástico en los consumos pasivos de celdas de batería y los escrutinios feroces de privacidad para erradicar el ecosistema criminal subterráneo en la Google Play Store, Android 12 y revisiones superiores desterran esta modalidad autodeshabilitando su soporte pasivo sin foco para acceder a la cola, anulando y forzando bloqueos.⁴⁰

Capa 3: FAB tap — La solución siempre disponible

Si las vías anteriores mueren o si el consumidor aborrece ceder control sistemático u omnipotente, la arquitectura regresa y descansa orgánicamente bajo el escrutinio explícito o empuje táctil manual (Fallback persistente), vehiculizado en forma de un superpuesto Botón de Acción Flotante omnipresente e indestructible (Persistent Floating Action Button — FAB, orquestado en ClipOverlayManager.kt).

Cuando la pulsación física humana incide y presiona la esfera superpuesta, el motor del sistema despacha al núcleo gráfico una solicitud imperativa de instanciar un contenedor incoloro (SendClipActivity). El "truco" de explotación radicado sobre esta actividad transparente consiste en que irrumpe en escena asumiendo un rol agresivo transitorio. En un asalto relámpago e imperceptible a nivel de nanosegundos al visor principal (robando el foco inmaculado a cualquier subrutina previa en ejecución), Android detecta y otorga el derecho incondicional basándose en el parámetro de ciclo de vida atómico onWindowFocusChanged(true).⁴² De este modo, la restricción de Android Q de que solo aplicaciones de cara visible y con foco activo pueden acceder al API del portapapeles se satisface legalmente.⁴²

El puente queda temporalmente abierto. Tras sorber el flujo del portapapeles y despachar la señal inalámbrica subyacente al framework de WebSocket, la silueta invisible del cliente parece prematuramente devolviendo el comando al entorno central subyacente. En escenarios patológicos donde cuellos de botella u omisiones caprichosas del gestor gráfico originan

silencios eternos o el método de enfoque de la interfaz no cristaliza debidamente, la subrutina desata una red de retención perentoria y agresiva fijando un retraso rígido u umbral de 200ms tras el que procede y consuma la purga obligatoria (un forzado intento final asimétrico) para liberar y clausurar las hebras de hilo principal sin bloquear interacciones.⁵⁸

Anti-echo y debounce explicados

La persistente necesidad de mantener puentes de diálogo asíncronos y perpetuos interrumpiendo nodos remotos propicia una anomalía parasitaria endémica e infernal en la ciencia de sistemas unificados y sincronizados a gran escala: las cadenas de retroalimentación en bucle y resonancias infinitas (Loop Echo).

Si el sistema macOS capta a través del ratón la selección copiada "Hola", automáticamente expide vía red una alerta JSON hacia el terminal Android. Como reacción incondicional, Android sobrescribe en sus memorias la cadena "Hola". Al alterarse físicamente el bloque temporal del registro portapapeles local, los oyentes pasivos del dispositivo se ven inmersos a disparar ciegamente la alerta en segundo plano, informando que una "modificación reciente o nueva ha ocurrido en el cliente", retornando la variable intacta hacia macOS, quien a su vez sobrecribiría y repetiría este proceso circular extenuando procesadores, redes e incrementando temperaturas ininterrumpidamente.

El motor **anti-echo** aborda el dilema insertando celdas de cuarentena asíncronas de naturaleza de retardo mudo (Mute Flags) tras eventos programáticos. En esencia, cuando una instrucción exógena inyectada por la base de red de macOS fuerza la impresión o sobrescritura local en el sistema del teléfono, Android engatilla inmediatamente un temporizador pasivo de retención y silenciamiento ciego programado durante 2 segundos en el receptor de la cola de eventos. Cualquier cambio colateral notificado durante el ciclo se descarta.

Conjuntamente, los marcos operativos API y sus algoritmos interpretativos o predicativos tienden a enredarse en avalanchas de alertas duplicadas (OnPrimaryClipChangedListener) causadas cuando rutinas paralelas de lectura porciones visuales (clasificaciones de aprendizaje para ofrecer búsquedas automáticas, URL o números embebidos) o formatos mixtos repiten notificaciones asimétricas simultáneas en colisiones atómicas al mismo milisegundo.⁵⁹ El algoritmo de apaciguamiento de rebotes iterativos (**Debounce logic**) instauro en contención una presa asíncrona de duración transitoria fijada a un segundo, aplazando ejecuciones recurrentes redundantes o transacciones asíncronas para ensamblarlas o purgarlas, liberando en última instancia la más reciente de forma consolidada para impedir ahogamientos de tráfico WebSocket.

9. Flujos de datos completos — De punta a punta

Los esquemas ASCII que siguen demuestran los caminos críticos (happy paths) de la transferencia del conocimiento y los blindajes de criptografía, ilustrando los fracasos sistemáticos ante colisiones inyectadas.

Flujo 1: Android envía texto al Mac (vía FAB tap)

|

|-> Despliegue atómico inyecta foco gráfico (SendClipActivity)
|
v
[Autorización Otorgada] -> Condición OS validada: El proceso detenta Foco Activo [API Q+]
|
|-> Extracción del flujo (Payload string extraída por ClipboardManager)
|
v
-> Ensamblaje Payload: {"type":"text","data":"..."}
|
|-> [Invocación Cripto] El HMAC-SHA256 firma JSON usando Timestamp dinámico
| y Pairing Secret inyectado.
v
|
|-> Valida integridad asimétrica blindando MITM e Inyecciones Router
v
|
|-> Procesamiento asíncrono (Hummingbird/Swift).
|-> Desentrañando encriptación y desencapsulando HMAC para validación temporal/bits.
| (*Ruptura: Si el timestamp fluctúa $\pm 60s$ o un bit ha sido reconfigurado en tránsito, el paquete se purga*)
v
[Ecosistema Nativo Mac] -> NSPasteboard inyecta en registro central.

Flujo 2: Android envía texto al Mac (vía auto-send / Shizuku)

Usuario selecciona y ejecuta 'Copiar' en una app de 3os.
|
v
|-> Percibe alteración sin impedimentos de sistema al operar con privilegios ADB nivel núcleo.
|
v
|-> Filtrado pasivo de validación: ¿Periodo Anti-echo habilitado? Si es No, continúa.
|-> Retención asíncrona de cadencia programada: 1000ms Debounce finalizados.
|
v
|-> Expide petición WebSocket/HTTP agregando Header {Authorization: Bearer } + {HMAC Firmado}
|
v
|-> Hash Verification: Coteja Hash (SHA-256) del Bearer Token recibido contra el registrado en Base local.
| (*Ruptura: Token inválido deniega en capa superficial y no malgasta procesador*)
|-> Coteja HMAC contra el Pairing Secret de sesión persistida.

v

[Ecosistema Nativo Mac] -> Reemplazo asíncrono automatizado (NSPasteboard actualizado).

Flujo 3: Mac envía texto a Android (NSPasteboard watcher → WebSocket)

[Evento Físico] Usuario activa evento teclado Cmd+C (macOS).

|

v

El registro universal subyacente NSPasteboard altera su contador de índice mutable de iteraciones (changeCount).

|

-> Demonio pasivo NSPasteboard Watcher asimila el latido de alteración de índices y absorbe contenido.

v

-> Discrimina el catálogo de sockets retenidos del pool WebSocket conectados y vinculados por la asociación de los Token válidos.

|

v

-> Remisión de Carga Útil Json { "action": "set_clip", "payload": "..." } blindado a los clientes.

v

-> Intercepción asíncrona por procesos en Kotlin, despachando subrutina incondicional y preventiva:

| *Engatillado de silenciador asíncrono de interrupciones "Anti-Echo" de Mute (2000ms)*

|

v

-> Comando imperativo setPrimaryClip() al subsistema ClipboardManager de Android (Escudado en permisos transitorios heredados por Shizuku o forzados visualmente).

Flujo 4: Mac envía imagen a Android (Notificación rich con thumbnail)

Atajo Cmd+C captura recuadro binario o imagen bruta (macOS).

|

v

[Motor Polling Interno] El NSPasteboard Watcher audita descriptores de formato e identifica cadena de metadatos asociada a clases TIFF/PNG incrustadas en portapapeles temporal.

|

v

-> El servidor genera compresión destructiva reescalada y aligera en base64 una vista reducida tipo miniatura o Thumbnail preview (e.g. 200x200px).

-> Notifica por red WebSocket o ping pasivo al móvil Android del evento binario masivo próximo.

v

-> Kotlin parsea alerta asíncrona e inyecta y despacha al manejador visual del cliente

| una Notificación Rich asimétrica con incrustación directa visual (Centro Notificaciones).

|

| (Nota: Retención Inteligente: Un archivo masivo no satura anchos de banda, redes móviles, | ni memorias heap RAM en background si el usuario jamás va a pegar dicha imagen gigante | en su aplicación en uso)

v

[Operación Humana y Extracción Final]

|-> El humano interactúa sobre la miniatura expansiva: Android reanuda conexión, dispara

| GET HTTP a API para volcado de Buffer crudo masivo HD final con firma y Bearer validado.

Flujo 5: Android envía imagen al Mac

El humano navega aplicaciones fotos y opera interactuando

sobre el marco visual con Compartir... o Share Intent nativo de la capa gráfica Android.

|

v

|-> La hebra secundaria del proceso absorbe la resolución física local de URI binario en base a | permisos visuales y permisos Android (READ_EXTERNAL_STORAGE).

|

v

|-> Se conforma y amalgama el tren binario multipart/form-data.

|-> El emisor OkHttp ancla y estampa en el Header tanto el Bearer Token del emparejamiento | como la rúbrica y sello de integridad dinámico HMAC-SHA256 validando tamaño de partes.

v

Encriptación de Bloque Completa de extremo a extremo TLS.

|

v

|-> Volcado a disco temporal y ensamblado de piezas, cotejo hash temporal validado en ruta.

|-> Integración y absorción por capas gráficas NSPasteboard para exponer el elemento

| vía formatos NSURL o NSPasteboardTypeTIFF disponibles globalmente en memoria viva del portátil.

10. Threat model — ¿De qué protege ClipSync y de qué no?

La criptografía es una constante de compromisos y limitaciones dictadas por la física del hardware y la negligencia humana.

Ataques que ClipSync resiste (con justificación técnica):

Categoría Amenaza	Capacidad de Mitigación Criptográfica y Base Técnica Subyacente
Ataque MITM (En LAN pública o abierta)	Neutralización y blindaje de túnel forzado y abrupto merced a la inyección dinámica de la

	huella técnica y las anclas SPKI Pinning, validando asimétricamente cada milisegundo que el interceptador o router suplantador fracase rotundamente en imitar y portar la clave privada criptográfica asimétrica correcta que coincida biunívocamente con el PIN local extraído originariamente en el pregonero (mDNS).¹
Replay Attacks (Repeticiones Maliciosas temporales)	Invalidación sistemática obligatoria y persistente inyectando la cadena binaria de estampas atómicas de tiempo cronológico UNIX dentro de los esquemas HMAC.²⁰ La tolerancia rígida ± 60 segundos y la firma matemática de validación destrozan manipulaciones.
Tampering (Manipulaciones de Inyección In-Transit)	Envoltorio irrompible mediante los sellos generadores de Integridad HMAC, imposibilitando adivinar, rehacer firmas o transmutar bytes en JSON asíncronos debido al aislamiento y desconexión hermética de las llaves secretas alojadas en enclaves hardware. La protección frente al descubrimiento forense evita deducción algorítmica constante.²⁷
Ataques de Conexión / Acceso Incondicionado	Interceptado y descartado gracias a la inclusión estricta en protocolos superficiales de validaciones Bearer Tokens para transacciones que se evalúan sin quemar memoria con verificaciones irreversibles de un único sentido (Hashes) frenando las avalanchas iniciales en base al TOFU original.
Enumeración Computacional del Apretón o Reutilización	Extinción garantizada matemáticamente y algorítmica al instaurar y aniquilar las combinatorias efímeras tras un brevísimo ciclo de vida de (300 segundos), la aleatorización base y deshabilitar reusos físicos.¹⁴
Extracción Pasiva Forense Local y Bloque de Discos Inactivos	Refugio insondable gracias a atar las cadenas asimétricas asíncronas en las estructuras modulares de hardware criptográfico integrado (Keychain Secure Enclaves y Android FBE/TEE), requiriendo decodificaciones inviables por software si la huella biológica / biometría atada al UID de las

	placas base matrices no opera o decodifica al unisono. ³⁰
--	--

Ataques que ClipSync NO resiste (Limitaciones e impotencias por honestidad técnica del modelo):

Fallos Estructurales / Inyecciones Asimétricas	Análisis Crítico y Reconocimiento Teórico Funcional
Dispositivos Android con alteraciones Kernels de superusuario (Root) y Explotaciones de Día Cero (Zero-Day) al núcleo	Si un intruso enraiza o un malware logra privilegios omnipotentes tipo System/Root eludiendo SELinux asincrónicamente o por error de flasheos, se abre la facultad técnica absoluta del espionaje en los canales y áreas vivas de memoria RAM del framework asíncrono, permitiendo inyectar variables de lectura, asaltar la envoltura en vivo o absorber y falsificar tokens temporales decodificados en el bus al operar sin protección real ni barreras modulares de aislamiento a la lógica del puente.
Compromisos e inyecciones de credenciales Keychain maestras macOS o contraseñas explícitas al portador	La lógica física del sistema ampara al usuario local autenticado. Si se roban las contraseñas numéricas originarias del administrador y se insertan manual o remota vía VNC o Backdoors interactivos, el motor de control Secure Enclave / SQLite desglosará legal y plácidamente el contenedor de bases de datos local desenscriptando secretos HMAC de facto.
Confiscación o Asaltos Físicos Locales a Estaciones o Móviles con Pantalla Desbloqueada	Las capas criptográficas subyacentes operan para salvaguardar y retener bloqueos en base a candados. Una irrupción biológica o un asalto físico, extrayendo los teléfonos en estado operativo e interfaces libres y conectadas asimétricamente a redes sin apagarse anula instantáneamente cada precepto de validación delegando al asaltante permisos para ver o actuar y emitir los datos.
Fugas e intervenciones de permisos y vectores Shizuku en el entorno inter-aplicaciones locales	Permitir de manera incontrolada delegar el manto de autorizaciones de comandos superusuario a aplicaciones maliciosas extrañas, orillará y facultará asincrónicamente al agente intruso el control ciego total de la red ADB-Shizuku, brindando libre tránsito

	asimétrico al malware para desglosar pasivamente cualquier proceso portapapeles con los mismos credenciales que benefician a la lógica legítima nativa. ⁴⁹
Brechas en Componentes Estructurales o librerías de base (Zero-Days sobre CryptoKit, NIOSSL, Android OkHttp, Frameworks)	El diseño criptográfico de la aplicación y el flujo operativo asume sin refutaciones posibles la rectitud técnica y ausencia nula de desbordamientos de buffer u obsolescencia inherente a las bibliotecas base que implementan protocolos estándar y de matemática en curvas P-256; un fallo subyacente en el corazón del sistema operativo destruye irremediablemente las previsiones. ¹²
Exfiltración Corporativa Voluntaria (Escapes y DLP nativo asimétrico)	La funcionalidad matriz está concebida, modelada y ejecutada estructuralmente para desplazar y emparejar sin bloqueos archivos o textos locales. Si se configura explícitamente ClipSync como puente autorizado interred por empleados infieles a terminales ajenos para copiar llaves corporativas, actuará en franca vulnerabilidad perenne y abierta frente a normativas estrictas corporativas de control o fuga de datos masivas (DLP - Data Loss Prevention).

Comparativa frente a servicios paralelos centralizados en la nube y paradigmas Cloud (Pushbullet / Clipt):

Las metodologías de sincronización sustentadas arquitectónicamente en la intermediación Cloud (como las operativas asíncronas detrás de aplicaciones preexistentes dependientes de servidores) son susceptibles a amenazas globales que una red P2P o modelo LAN invertido tipo ClipSync erradica conceptualmente. Las entidades de nube se enfrentan a escenarios de filtración masiva pasiva de bases de datos centralizadas por intrusiones de red en los silos, asimilación y vulnerabilidad pasiva de servidores ante empleados corporativos con motivaciones dudosas y la potencial obediencia asimétrica o entrega de bases de datos a peticiones coactivas de instituciones mediante citaciones legales genéricas u órdenes de investigación de inteligencia (Subpoenas o espionaje pasivo incontrolado masivo de NSA, etc.). Aun así, la naturaleza monolítica local P2P arrastra deficiencias insalvables operativas que la conectividad a un centro general provee amistosamente, destacando la disponibilidad ubicua incondicionada bajo sistemas integrados de Autenticación OAuth 2.0 (Google ID), dispensando el uso asimétrico a los usuarios que se niegan a orquestar un túnel de malla asíncrona WireGuard y rechazan configurar un modelo superpuesto frágil o redes de VPN perennes. Mientras las soluciones comerciales centralizadas exigen menos fricción inicial pero absorben pasivamente derechos y control del histórico visual del consumidor y sus textos portapapeles

diarios, un modelo invertido y criptográficamente hostil externaliza el dolor del enrutamiento asíncrono y los arreglos de IP sobre VPNs hacia un usuario técnicamente dispuesto pero paranoico en el contexto de sus metadatos e inviolabilidad perimetral.

11. Deuda técnica y decisiones de diseño

La asimetría del proyecto despliega en su concepción profundos vacíos técnicos elegidos conscientemente, fricciones topológicas o áreas de inmadurez predeterminadas a la resolución en escalabilidades ulteriores. Todo diseño arquitectónico se enmarcó dentro de un compromiso dictado por los ecosistemas imperantes y recursos o frameworks asimétricos. Porciones críticas de la lógica del sistema operan orgánicamente a nivel de código subyacente pero carecen perversamente de su emparejamiento visual o conexión (cableado) final con los menús de la interfaz interactiva. La base de datos de control interno de macOS almacena de forma asimétrica y segmentada un catálogo histórico tabular aislando los Bearer Tokens por dispositivos independientes e identifica variables que amparan y viabilizan teóricamente la revocación táctica y microscópica de uno de los terminales comprometidos y robados sin deshabilitar al enjambre de terminales funcionales restantes. Lamentablemente, la usabilidad gráfica fue abortada y omitida de cara a la implementación en su estado original, castigando a los usuarios asimétricamente a invalidar y presionar botones que desencadenan el purgado de raíz de todo el "Pairing-secret" atómico al completo y que orquestará inevitablemente repeticiones engorrosas y ritos TOFU de toda la red desde cero.

Las omisiones interplataforma son endémicas en el concepto actual; Windows e iOS (que posee aislamientos intrínsecos paralelos restrictivos y barreras de procesos de fondo insalvables asimétricos con requerimientos forzosos Apple ID sin universalidad mDNS y restricciones a frameworks externos limitando la operatividad general e interactividad manual) están intencionadamente excluidos, limitando drásticamente un proyecto de sincronía que paradójicamente ansía una transversalidad pura. Asimismo, fallan implementaciones recurrentes obligatorias o rotaciones automáticas criptográficas de Tokens que desechen claves estáticas tras meses ininterrumpidos y garanticen re-anclajes (tipo OAuth Refresh Tokens para invalidaciones caducas) dejando expuesto infinitamente al usuario ante secuestros silenciosos a menos de revocar el Token o el ecosistema de supresión de redes manuales. Por carencias logísticas y sobrecalentamientos térmicos de buffers Android en envíos asíncronos en masa o caídas, la sincronía plena asimétrica o copiado de "Ficheros Completos Multi GBs" carece de transacciones nativas y obligaría a rehacer de raíz la mensajería HTTP adoptando streams por WebRTC fraccionado P2P en el flujo pasivo para repeler la inundación de heaps en la memoria viva (Out-Of-Memory errors en máquinas virtuales Android con cuotas asíncronas rígidas por app).

Desde el plano netamente de enrutamiento y redes superpuestas, se arrastra estructuralmente y por tiempo indefinido el "Bug" e incompatibilidad letal inherente a las implementaciones directas UDP multidifusión sobre los puertos mDNS en infraestructuras VPN estáticas en capa 3. La VPN Tailscale desecha en bruto pasivamente los dominios multicasts en su trayecto cifrado al basar su nodo y control atómico local asimétrico sobre topologías lógicas cerradas descartando peticiones Bonjour o dominios broadcast.⁵ Este fallo topológico asimétrico forzó a

instituir las metodologías manuales primitivas pero funcionales como la inclusión e inserción forzada IP al sistema y anclar cruces extras de verificación del enrutamiento final al socket y la máquina Mac al fallar el fingerprint local de SPKI pre-transferido (El "Extra Check" de consistencia de IP vs Certificados a través de librerías nDPI implementadas o similares).¹¹ Para curar este fallo base en inter-redes VPN y obviar los tecleos arcaicos IP, la implementación demandaría un acoplamiento asimétrico explícito al sistema registrador central asíncrono y los enrutadores "MagicDNS" subyacentes e interconectados del entorno interno de configuración subyacente WireGuard/Tailscale.

Sin embargo, quizás la decisión técnica asimétrica base y de diseño primigenia con ramificaciones de mayor impacto haya sido la elección y cimentación rígida alrededor del servidor de capa web y framework **Hummingbird 2.x**. Este motor subyacente asíncrono fue completamente derribado desde sus simientes para renacer estructurado integralmente en la concurrencia nativa, estructurada y atómica del futuro sistema Swift y sus nuevas visiones de memoria segura asimétrica.⁶⁰ Tras una gestación intermitente de año y medio asimilando e implantando más de 35 proposiciones del diseño y evolución (Evolution Proposals), la versión renovada del ecosistema HTTP sustituye asimétrica y fundamentalmente el obsoleto concepto laberíntico atómico basado originariamente en Promesas y delegaciones reactivas (EventLoopFutures acoplados en su motor NIO subyacente de Apple) en favor orgánico de implementaciones modulares nativas mediante `async/await`, reestructurando los contextos, desintegrando variables pasivas sin asilamiento estructurado atómico (TaskGroup formales en servicios) y promoviendo el apagado ordenado asíncrono (swift-service-lifecycle v2).⁶⁰

Los beneficios subyacentes atómicos logran legibilidades, purezas lógicas y mantenimientos asincrónicos superiores contra fugas de memoria o variables compartidas por colisiones de red.⁶⁰ El compromiso y peaje insalvable e inmensamente hostil adoptado es la exigencia nativa inquebrantable asimétrica a integrarse utilizando lenguajes base vinculados estrechamente (y atados monolíticamente por ecosistemas dependientes directos) con la integración y prerrogativas asincrónicas incrustadas inexorablemente al compilador de versión asíncrona pre-Swift 6 y su sistema perimetral de validaciones estrictas. Debido a la falta de adaptadores de portabilidad y APIs primitivas limitadas por el sistema base inferior, el framework, por defecto impone dictatorialmente compilaciones asincrónicas cuyo suelo operativo y base de despliegue inferior arranca y se ancla implícitamente al ecosistema base de **macOS 14 (Sonoma)** o superior (restando y arrinconando la retrocompatibilidad atómica con terminales de base como Monterey/Ventura).⁶⁰

Este límite y tradeoff técnico destierra y mutila irremediabilmente del campo de adopción asimétrico base del proyecto a la legión y parque activo pasivo total de miles de usuarios atrapados atómicamente y sin actualizaciones oficiales subyacentes con ordenadores Intel y MacBooks base históricos de versiones del 2016 o el 2017 por carencias puras de silicio. Para un entorno donde las prioridades asimétricas operativas asincrónicas imponen las latencias mínimas y seguridades atómicas del código y la red, el sacrificio e inmolación drástica o eliminación total base de la masiva retrocompatibilidad del producto representa el coste asumible final o consecuencia natural ante los beneficios implacables e irrenunciables que derivan asincrónicamente de fundar y modelar el puente perimetral y el enrutador JSON Swift

bajo una modernidad atómica implacable y de seguridad robusta e inquebrantable a largo plazo de este framework.

Obras citadas

1. DNS over TLS (DoT) SPKI Fingerprint Pinning Issue Debugging - GitHub Gist, fecha de acceso: abril 27, 2026, <https://gist.github.com/Oxdevalias/e5430349a3e6e5feb347f8a373877f4e>
2. DNS over TLS - » RFC Editor, fecha de acceso: abril 27, 2026, <https://www.rfc-editor.org/rfc/rfc7858.txt>
3. Multicast DNS-Based Service Discovery for Encrypted DNS Services - IETF, fecha de acceso: abril 27, 2026, <https://www.ietf.org/archive/id/draft-liu-add-dnssd-edns-01.html>
4. SSL pinning. How to make it right. | by Oleksandr Stepanov - Medium, fecha de acceso: abril 27, 2026, <https://oleksandr-stepanov.medium.com/ssl-pinning-how-to-make-it-right-ecc5c9844215>
5. Subnet routers · Tailscale Docs, fecha de acceso: abril 27, 2026, <https://tailscale.com/docs/features/subnet-routers>
6. Tailscale and the OSI model, fecha de acceso: abril 27, 2026, <https://tailscale.com/docs/concepts/tailscale-osi>
7. tailscale ain't a good choice when it comes to mDNS - Reddit, fecha de acceso: abril 27, 2026, https://www.reddit.com/r/Tailscale/comments/1ht6t4q/tailscale_aint_a_good_choice_when_it_comes_to_mdns/
8. Support mDNS for name and service resolution · Issue #1013 · tailscale/tailscale - GitHub, fecha de acceso: abril 27, 2026, <https://github.com/tailscale/tailscale/issues/1013>
9. FR: Support for general purpose multicast · Issue #11134 · tailscale/tailscale - GitHub, fecha de acceso: abril 27, 2026, <https://github.com/tailscale/tailscale/issues/11134>
10. mDNS is so cool! | Muhammad Fareez Iqmal | Software Engineer, Maker | Malaysia, fecha de acceso: abril 27, 2026, <https://iqfareez.com/blog/mdns-is-cool>
11. Releases · ntop/nDPI - GitHub, fecha de acceso: abril 27, 2026, <https://github.com/ntop/ndpi/releases>
12. Self-Signed and Pinned Certificates in Go - fREW Schmidt's Foolish Manifesto, fecha de acceso: abril 27, 2026, <https://blog.afoolishmanifesto.com/posts/golang-self-signed-and-pinned-certs/>
13. Certificate and Public Key Pinning | OWASP Foundation, fecha de acceso: abril 27, 2026, https://owasp.org/www-community/controls/Certificate_and_Public_Key_Pinning
14. Password Entropy Calculator, fecha de acceso: abril 27, 2026, <https://www.omnicalculator.com/other/password-entropy>
15. Password strength - Wikipedia, fecha de acceso: abril 27, 2026, https://en.wikipedia.org/wiki/Password_strength

16. Entropy requirements vs. length requirements · Issue #1218 · OWASP/ASVS - GitHub, fecha de acceso: abril 27, 2026,
<https://github.com/OWASP/ASVS/issues/1218>
17. How should I calculate the entropy of a password? - Cryptography Stack Exchange, fecha de acceso: abril 27, 2026,
<https://crypto.stackexchange.com/questions/374/how-should-i-calculate-the-entropy-of-a-password>
18. "The Same PIN, Just Longer:" On the (In)Security of Upgrading PINS from 4 to 6 Digits - USENIX, fecha de acceso: abril 27, 2026,
<https://www.usenix.org/system/files/sec22-munyendo.pdf>
19. How to make it work Certificate pinning (HPKP) and self signed certificate in a local network? - Server Fault, fecha de acceso: abril 27, 2026,
<https://serverfault.com/questions/876798/how-to-make-it-work-certificate-pinning-hpkp-and-self-signed-certificate-in-a>
20. HMAC-SHA256 in Swift | Hashing and Validation Across Programming Languages, fecha de acceso: abril 27, 2026,
<https://mojoauth.com/hashing/hmac-sha256-in-swift>
21. HMAC-SHA224 in Swift | Hashing and Validation in Multiple Programming Languages, fecha de acceso: abril 27, 2026,
<https://ssojet.com/hashing/hmac-sha224-in-swift>
22. Timing attack against HMAC in authenticated encryption?, fecha de acceso: abril 27, 2026,
<https://security.stackexchange.com/questions/74547/timing-attack-against-hmac-in-authenticated-encryption>
23. HMAC-SHA256 in Kotlin | Hashing and Validation in Multiple Programming Languages, fecha de acceso: abril 27, 2026,
<https://ssojet.com/hashing/hmac-sha256-in-kotlin>
24. HMAC-SHA3-256 in Kotlin | Hashing and Validation in Multiple Programming Languages, fecha de acceso: abril 27, 2026,
<https://ssojet.com/hashing/hmac-sha3-256-in-kotlin>
25. What is the preferred way of comparing hmac signatures in Node? - Stack Overflow, fecha de acceso: abril 27, 2026,
<https://stackoverflow.com/questions/51486432/what-is-the-preferred-way-of-comparing-hmac-signatures-in-node>
26. Constant time equals - java - Stack Overflow, fecha de acceso: abril 27, 2026,
<https://stackoverflow.com/questions/37633688/constant-time-equals>
27. How could HMAC comparison ever not be constant-time in Python? - Stack Overflow, fecha de acceso: abril 27, 2026,
<https://stackoverflow.com/questions/55261040/how-could-hmac-comparison-ever-not-be-constant-time-in-python>
28. Equality Checking with Constant Runtime - Language Design - Kotlin Discussions, fecha de acceso: abril 27, 2026,
<https://discuss.kotlinlang.org/t/equality-checking-with-constant-runtime/15262>
29. HMAC-SHA256 in Swift | Hashing and Validation in Multiple Programming Languages, fecha de acceso: abril 27, 2026,

- <https://ssojet.com/hashing/hmac-sha256-in-swift>
30. The Secure Enclave - Apple Support, fecha de acceso: abril 27, 2026,
<https://support.apple.com/guide/security/secure-enclave-sec59b0b31ff/web>
 31. Operating system integrity - Apple Support, fecha de acceso: abril 27, 2026,
<https://support.apple.com/guide/security/operating-system-integrity-sec8b776536b/web>
 32. iOS Keychain vs Android Keystore: What Every Mobile Developer Should Know | by Musaddiq Ahmed Khan | Medium, fecha de acceso: abril 27, 2026,
<https://medium.com/@musaddiq625/%EF%B8%8F-ios-keychain-vs-android-keystore-what-every-mobile-developer-should-know-2f677e4acac0>
 33. Trusted Execution Environments / Secure Enclave | by Ranjit Murali - Medium, fecha de acceso: abril 27, 2026,
<https://medium.com/@rnjmurali2/trusted-execution-environments-secure-enclave-f41750d97a66>
 34. Android Keystore system | Security - Android Developers, fecha de acceso: abril 27, 2026, <https://developer.android.com/privacy-and-security/keystore>
 35. iOS Keychain vs. Android Keystore | AppSec Articles - Docs Portal, fecha de acceso: abril 27, 2026,
<https://docs.talsec.app/appsec-articles/articles/ios-keychain-vs.-android-keystore>
 36. Android Device Encryption vs iOS Device Encryption - Hexnode, fecha de acceso: abril 27, 2026,
<https://www.hexnode.com/blogs/android-device-encryption-vs-ios-device-encryption/>
 37. Implementing Remote Interface Using AIDL - ProTech Training, fecha de acceso: abril 27, 2026,
<https://www.protechtraining.com/blog/post/implementing-remote-interface-using-aidl-66>
 38. Let's Settle This For Once and For All: Android is just as Secure as iOS : r/Tech_Philippines, fecha de acceso: abril 27, 2026,
https://www.reddit.com/r/Tech_Philippines/comments/1oscfc7/lets_settle_this_for_once_and_for_all_android_is/
 39. Secure Clipboard Handling - Android Developers, fecha de acceso: abril 27, 2026,
<https://developer.android.com/privacy-and-security/risks/secure-clipboard-handling>
 40. Android Accessibility Suite Threat Protection | Guardsquare, fecha de acceso: abril 27, 2026,
<https://www.guardsquare.com/blog/protecting-against-android-accessibility-services-threats>
 41. Privacy changes in Android 10 | Android Developers, fecha de acceso: abril 27, 2026,
<https://developer.android.com/about/versions/10/privacy/changes#clipboard-data>
 42. How to access clipboard data programmatically in Android Q (10 ...), fecha de acceso: abril 27, 2026,

- <https://stackoverflow.com/questions/59903001/how-to-access-clipboard-data-programmatically-in-android-q-10>
43. Protecting Android clipboard content from unintended exposure | Microsoft Security Blog, fecha de acceso: abril 27, 2026,
<https://www.microsoft.com/en-us/security/blog/2023/03/06/protecting-android-clipboard-content-from-unintended-exposure/>
 44. Behavior changes: all apps | Android Developers, fecha de acceso: abril 27, 2026,
<https://developer.android.com/about/versions/12/behavior-changes-all>
 45. Android 12+ clipboard paste notification is a UX regression and needs a disable option, fecha de acceso: abril 27, 2026,
https://www.reddit.com/r/Android/comments/1q5finh/android_12_clipboard_paste_notification_is_a_ux/
 46. Behavior changes: all apps - Android Developers, fecha de acceso: abril 27, 2026,
<https://developer.android.com/about/versions/13/behavior-changes-all>
 47. Android 13's clipboard security protection trips up some apps - ZDNET, fecha de acceso: abril 27, 2026,
<https://www.zdnet.com/article/android-13s-clipboard-security-protection-trips-up-some-apps/>
 48. How to Elevate App Permissions using Shizuku library - XDA Developers, fecha de acceso: abril 27, 2026,
<https://www.xda-developers.com/implementing-shizuku/>
 49. What are you using Shizuku for now that Tasker support it natively? - Reddit, fecha de acceso: abril 27, 2026,
https://www.reddit.com/r/tasker/comments/1lvgbt/what_are_you_using_shizuku_or_now_that_tasker/
 50. Granting Permissions in Android 13 and 14 | Help Articles - AirData UAV, fecha de acceso: abril 27, 2026,
<https://app.airdata.com/wiki/Help/Granting+Permissions+in+Android+13+and+14>
 51. How to Access Data Folder on Android 11/12/13/14 Without Root (2024 Guide) - YouTube, fecha de acceso: abril 27, 2026,
<https://www.youtube.com/watch?v=gYARPZRrVXo>
 52. Shizuku - Keeps loosing running status : r/tasker - Reddit, fecha de acceso: abril 27, 2026,
https://www.reddit.com/r/tasker/comments/1mu6qn7/shizuku_keeps_loosing_running_status/
 53. How to Fix Shizuku Not Running on Non-Rooted Devices - YouTube, fecha de acceso: abril 27, 2026,
<https://www.youtube.com/watch?v=euw1A1ERphE>
 54. GitHub - RikkaApps/Shizuku-API: The API and the developer guide for Shizuku and Sui., fecha de acceso: abril 27, 2026,
<https://github.com/RikkaApps/Shizuku-API>
 55. Create an accessibility service - Android Developers, fecha de acceso: abril 27, 2026,
<https://developer.android.com/guide/topics/ui/accessibility/service>
 56. How to use onWindowFocusChanged() method? - Stack Overflow, fecha de acceso: abril 27, 2026,
<https://stackoverflow.com/questions/7924296/how-to-use-onwindowfocuschanged-method>

57. When to check the clipboard content in Android API 29 when app gets focus?,
fecha de acceso: abril 27, 2026,
<https://stackoverflow.com/questions/61732218/when-to-check-the-clipboard-content-in-android-api-29-when-app-gets-focus>
58. Diff -
3edd8f06cb5538d0f6cb7e9ca844237111802470^2..3edd8f06cb5538d0f6cb7e9ca844237111802470 - platform/frameworks/base - Git at Google - Android
GoogleSource, fecha de acceso: abril 27, 2026,
<https://android.googlesource.com/platform/frameworks/base/+3edd8f06cb5538d0f6cb7e9ca844237111802470%5E2..3edd8f06cb5538d0f6cb7e9ca844237111802470/>
59. ClipboardManager.OnPrimaryClipChangedListener | API reference ..., fecha de acceso: abril 27, 2026,
<https://developer.android.com/reference/android/content/ClipboardManager.OnPrimaryClipChangedListener>
60. Hummingbird 2, fecha de acceso: abril 27, 2026,
<https://hummingbird.codes/news/hummingbird-2>
61. What's new in Hummingbird 2? - Swift on server, fecha de acceso: abril 27, 2026,
<https://swiftonserver.com/whats-new-in-hummingbird-2/>
62. Swift on Server with Hummingbird 2 — MongoDB | by Szabolcs Toth | Medium,
fecha de acceso: abril 27, 2026,
<https://medium.com/@kicsipixel/swift-on-server-with-hummingbird-2-086cfccf4fae>
63. GitHub - hummingbird-project/hummingbird: Lightweight, flexible HTTP server framework written in Swift, fecha de acceso: abril 27, 2026,
<https://github.com/hummingbird-project/hummingbird>
64. Swift on Server with Hummingbird 2 — OracleNIO | by Szabolcs Toth | Medium,
fecha de acceso: abril 27, 2026,
<https://medium.com/@kicsipixel/swift-on-server-with-hummingbird-2-daefd3adf440>
65. Hummingbird | Documentation, fecha de acceso: abril 27, 2026,
<https://docs.hummingbird.codes/2.0/documentation/hummingbird/>
66. Introducing Hummingbird Category - Swift Forums, fecha de acceso: abril 27, 2026, <https://forums.swift.org/t/introducing-hummingbird-category/83576>